Question 1: Create a ranking of students based on score (highest first).

SQL Query:

```
USE sql_functions_db;

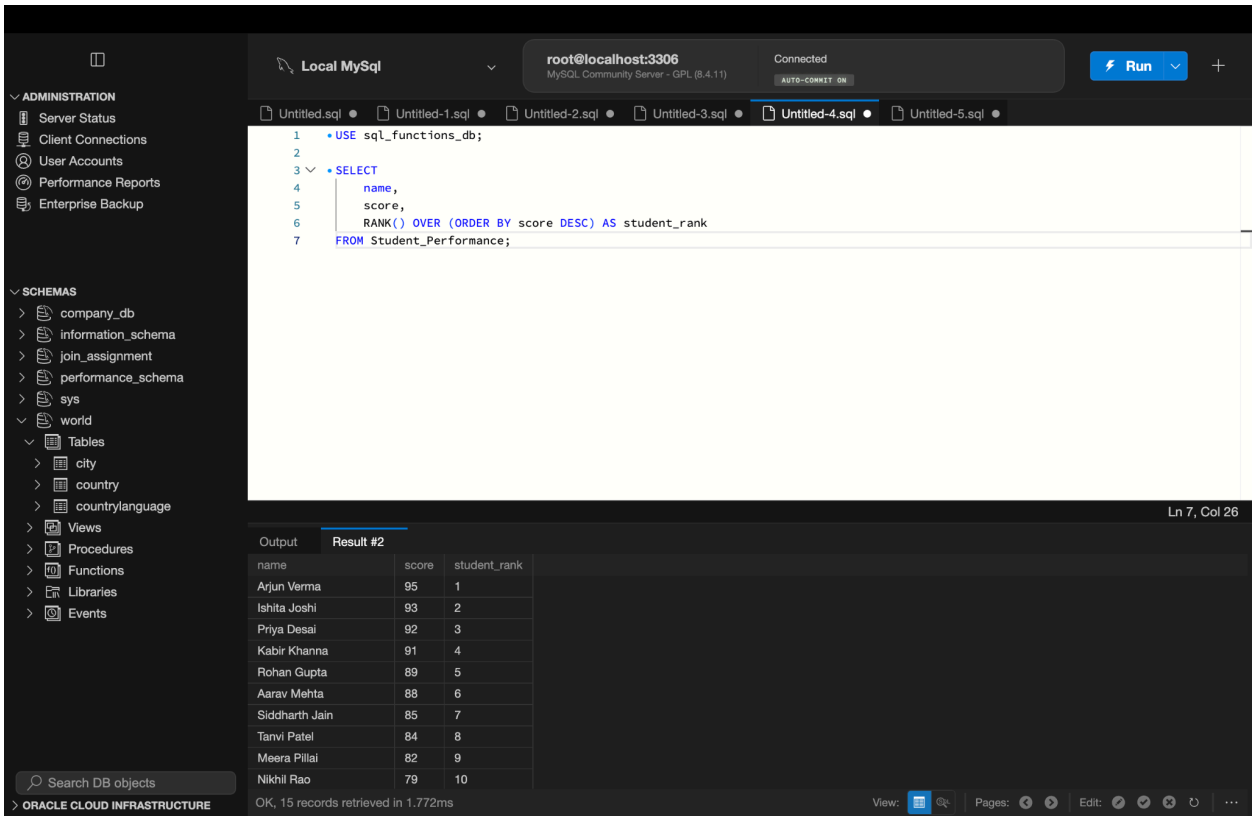
SELECT
    name,
    score,
    RANK() OVER (ORDER BY score DESC) AS student_rank
FROM Student_Performance;
```

Explanation:

The `RANK()` window function assigns a rank to each student based on their score. The `ORDER BY score DESC` clause places the highest score first.

Result: Arjun Verma secured the first rank with a score of 95.

Practical proof :



Output Result #2

name	score	student_rank
Aarav Mehta	88	6
Siddharth Jain	85	7
Tanvi Patel	84	8
Meera Pillai	82	9
Nikhil Rao	79	10
Riya Singh	76	11
Sneha Kulkarni	74	12
Dev Sharma	73	13
Yuvraj Rao	71	14
Ayesha Khan	67	15

OK, 15 records retrieved in 1.772ms

Question 2: Show each student's score and the previous student’s score (based on score order).

SQL Query:

```
USE sql_functions_db;

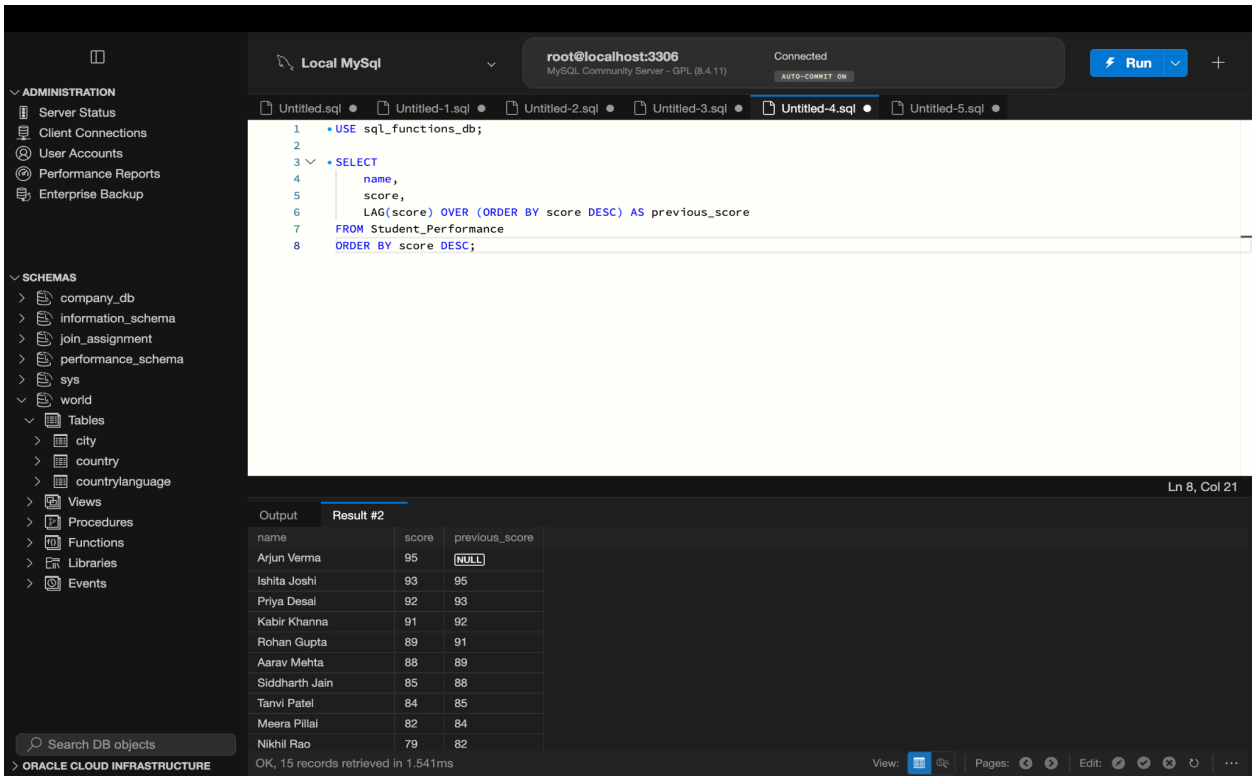
SELECT
    name,
    score,
    LAG(score) OVER (ORDER BY score DESC) AS previous_score
FROM Student_Performance
ORDER BY score DESC;
```

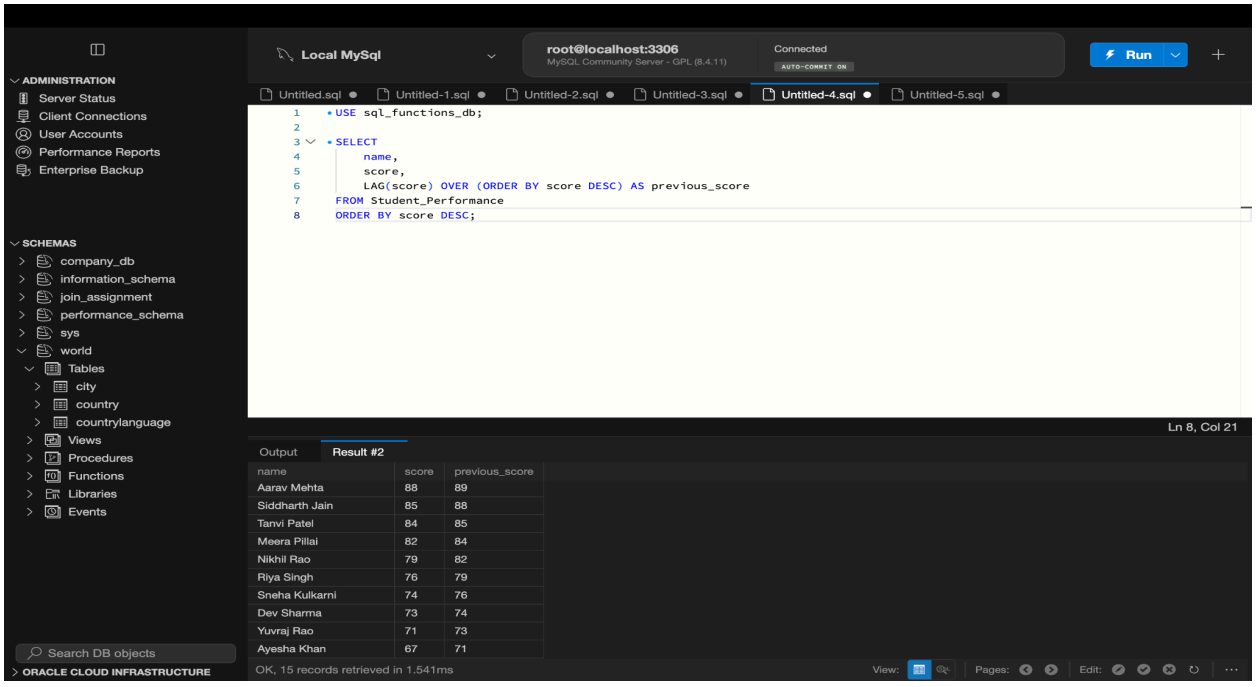
Explanation:

The `LAG()` function retrieves the score from the previous row according to the specified ordering. The highest-scoring student has `NULL` as the previous score because no row comes before that student.

Result: The query displays each student's score alongside the previous student's score, ordered from highest to lowest.

Practical proof :





Question 3: Convert all student names to uppercase and extract the month name from join_date.

SQL Query:

```
USE sql_functions_db;

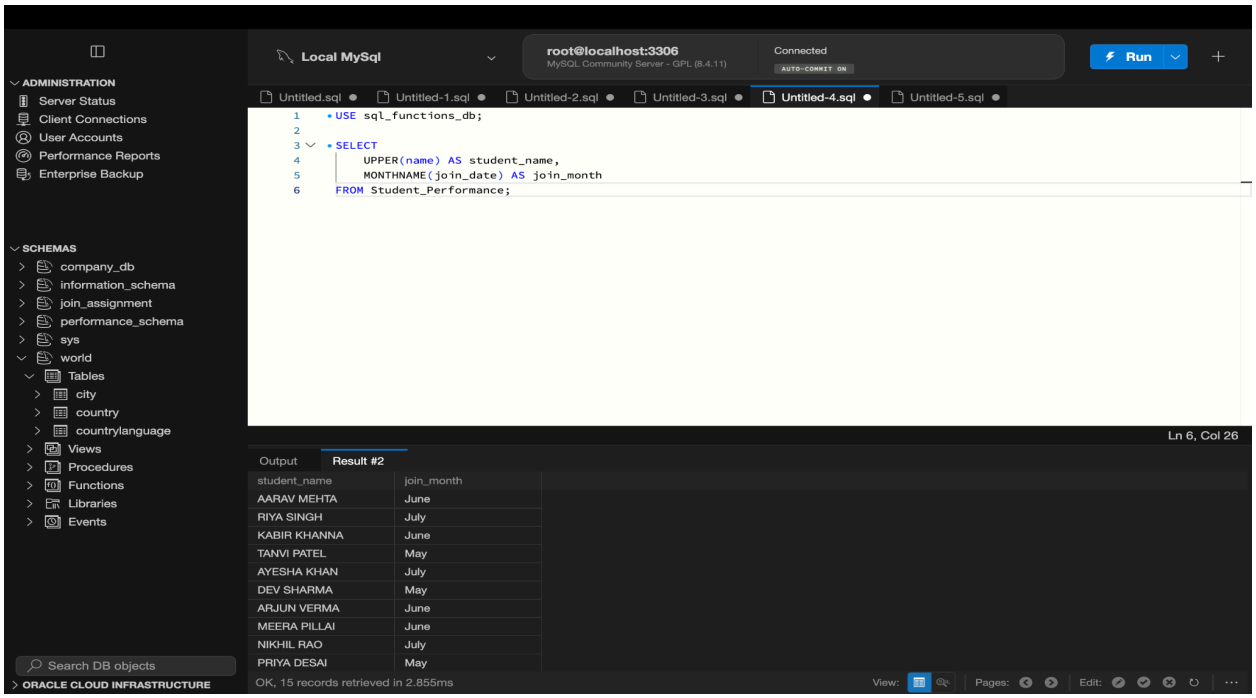

SELECT
    UPPER(name) AS student_name,
    MONTHNAME(join_date) AS join_month
FROM Student_Performance;
```

Explanation:

The `UPPER()` function converts student names into uppercase letters. The `MONTHNAME()` function extracts the month name from the `join_date` column.

Result: The query displays every student's name in uppercase along with the month they joined.

Practical proof :



PRIYA DESAI	May
SIDDHARTH JAIN	July
SNEHA KULKARNI	June
ROHAN GUPTA	May
ISHITA JOSHI	June
YUVRAJ RAO	July

Question 4: Show each student's name and the next student’s attendance (ordered by attendance).

SQL Query:

```
USE sql_functions_db;

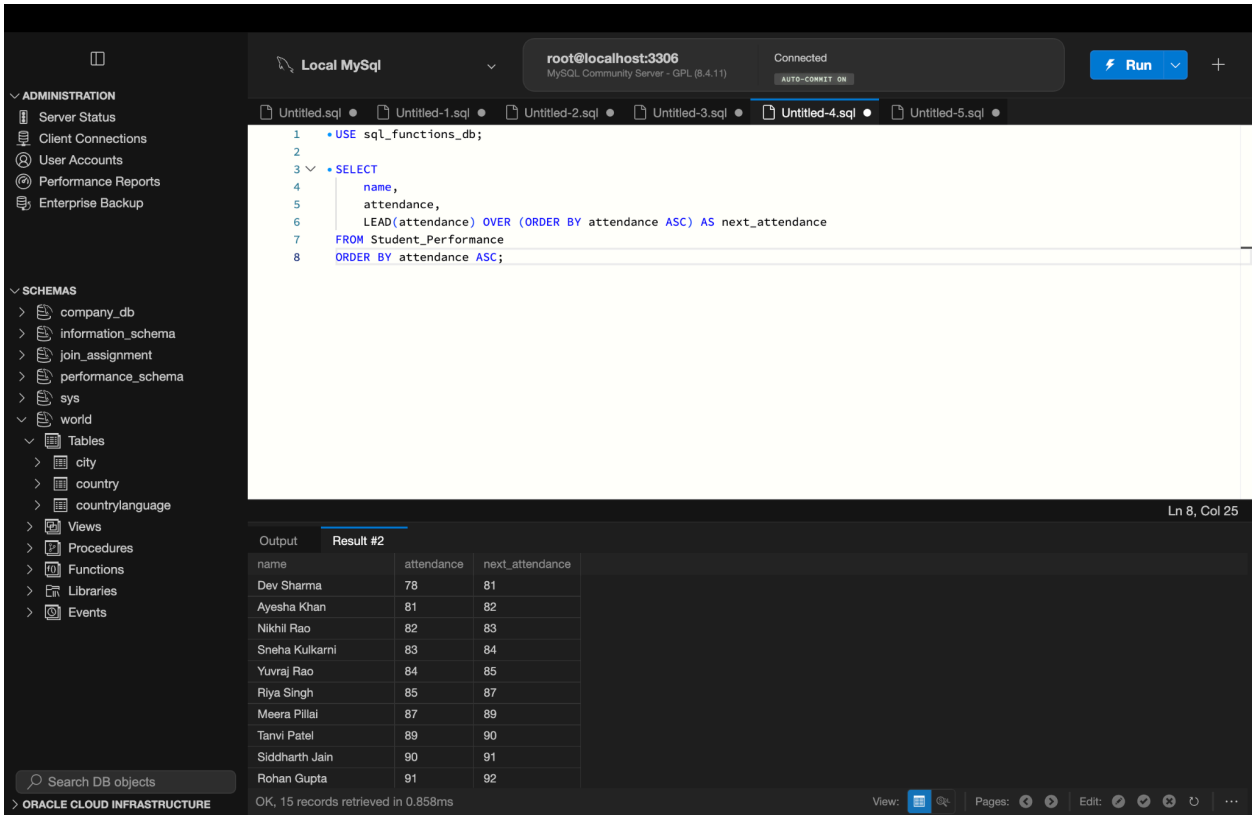

SELECT
    name,
    attendance,
    LEAD(attendance) OVER (ORDER BY attendance ASC) AS next_attendance
FROM Student_Performance
ORDER BY attendance ASC;
```

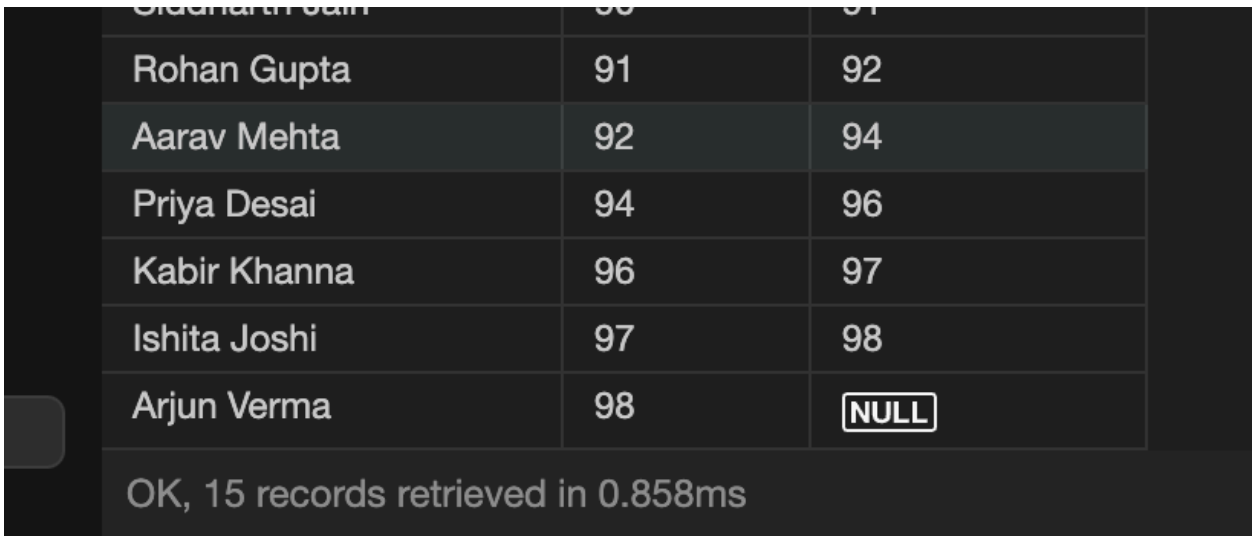
Explanation:

The `LEAD()` function retrieves the attendance value from the next row. Here, students are ordered by attendance from lowest to highest. The last student has `NULL` because there is no next row.

Result: The query displays each student's attendance and the next student's attendance.

Practical proof :





Question 5: Assign students into 4 performance groups using NTILE().

SQL Query:

USE sql_functions_db;

SELECT

name,

score,

NTILE(4) OVER (ORDER BY score DESC) AS performance_group

FROM Student_Performance

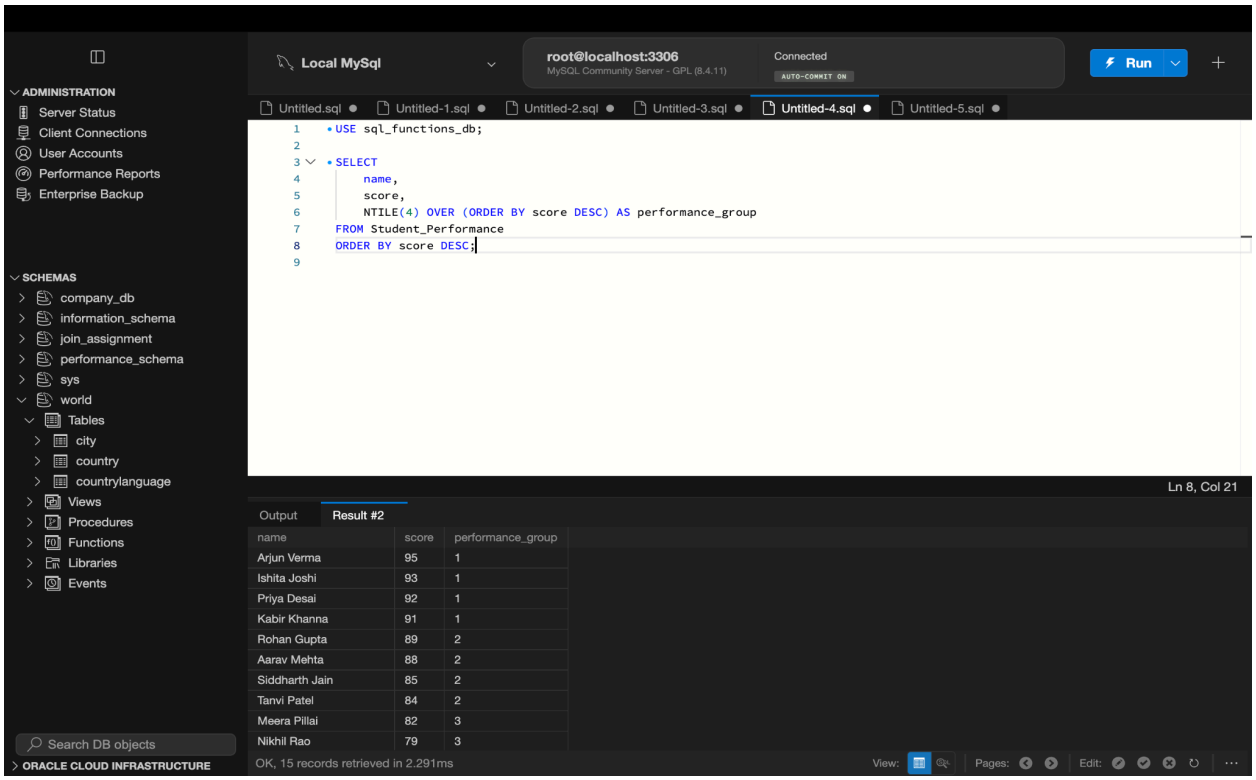
ORDER BY score DESC;

Explanation:

The `NTILE(4)` function divides the ordered rows into four performance groups. Students are sorted by score from highest to lowest before being assigned to groups.

Result: All 15 students are divided into four performance groups. The groups contain 4, 4, 4, and 3 students, respectively.

Practical proof :



Meera Pillai	82	3
Nikhil Rao	79	3
Riya Singh	76	3
Sneha Kulkarni	74	3
Dev Sharma	73	4
Yuvraj Rao	71	4
Ayesha Khan	67	4

OK, 15 records retrieved in 2.291ms

Question 6: For each course, assign a row number based on attendance (highest first).

SQL Query:

```
USE sql_functions_db;

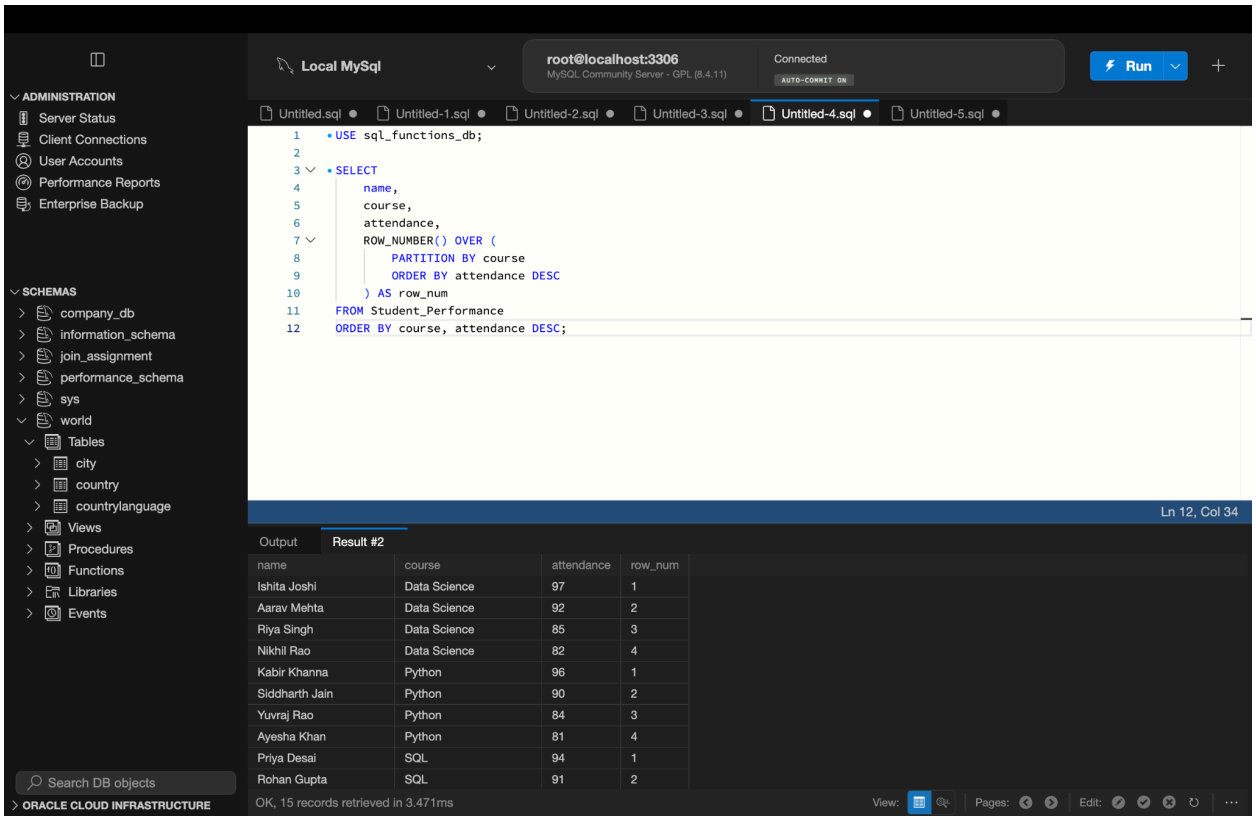

SELECT
    name,
    course,
    attendance,
    ROW_NUMBER() OVER (
        PARTITION BY course
        ORDER BY attendance DESC
    ) AS row_num
FROM Student_Performance
ORDER BY course, attendance DESC;
```

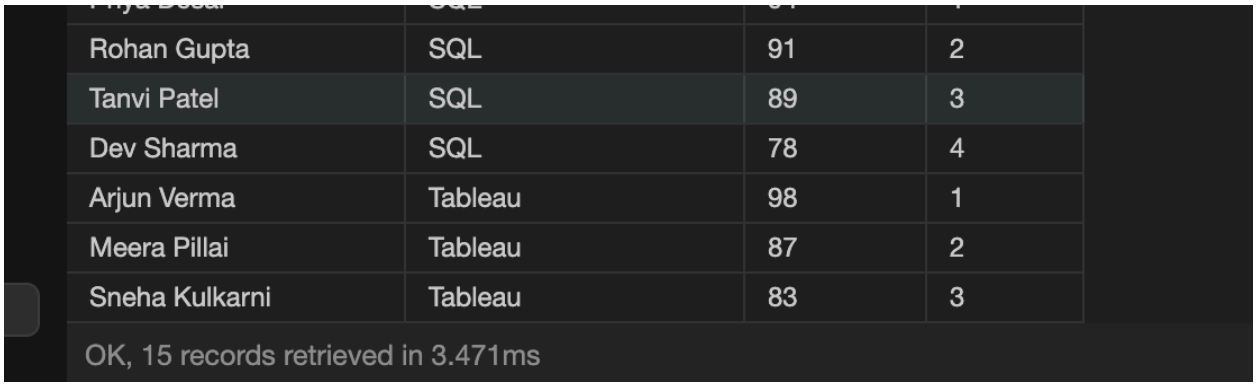
Explanation:

The `ROW_NUMBER()` function assigns a unique number to each student within their course. `PARTITION BY course` restarts the numbering for each course, while `ORDER BY attendance DESC` puts the student with the highest attendance first.

Result: Students are ranked separately within each course based on attendance.

Practical proof :





Question 7: Calculate the number of days each student has been enrolled (from join_date to today).

Assume current date = '2025-01-01'

SQL Query:

```
USE sql_functions_db;

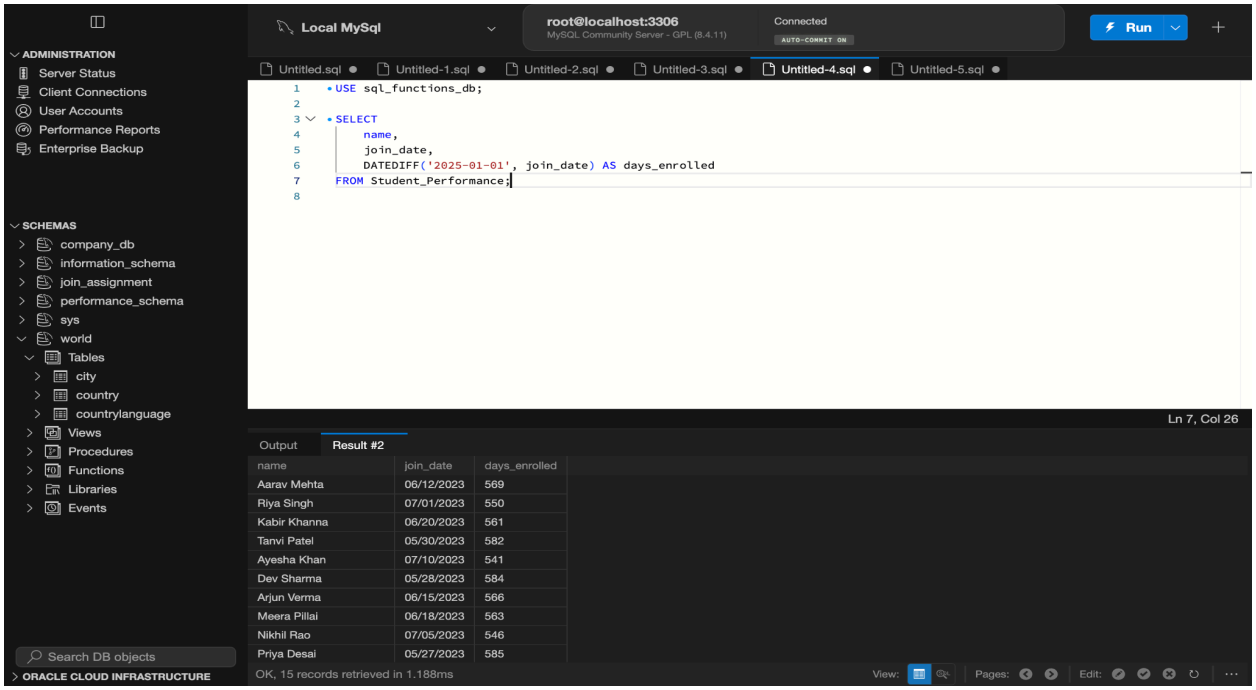

SELECT
    name,
    join_date,
    DATEDIFF('2025-01-01', join_date) AS days_enrolled
FROM Student_Performance;
```

Explanation:

The `DATEDIFF()` function calculates the number of days between two dates. The query uses January 1, 2025, as the current date, as specified in the assignment.

Result: The query calculates the number of days each student has been enrolled as of January 1, 2025.

Practical proof :





Priya Desai	05/27/2023	585
Siddharth Jain	07/02/2023	549
Sneha Kulkarni	06/10/2023	571
Rohan Gupta	05/25/2023	587
Ishita Joshi	06/25/2023	556
Yuvraj Rao	07/12/2023	539

URE

OK, 15 records retrieved in 1.188ms

Question 8: Format join_date as “Month Year” (e.g., “June 2023”).

SQL Query:

```
USE sql_functions_db;

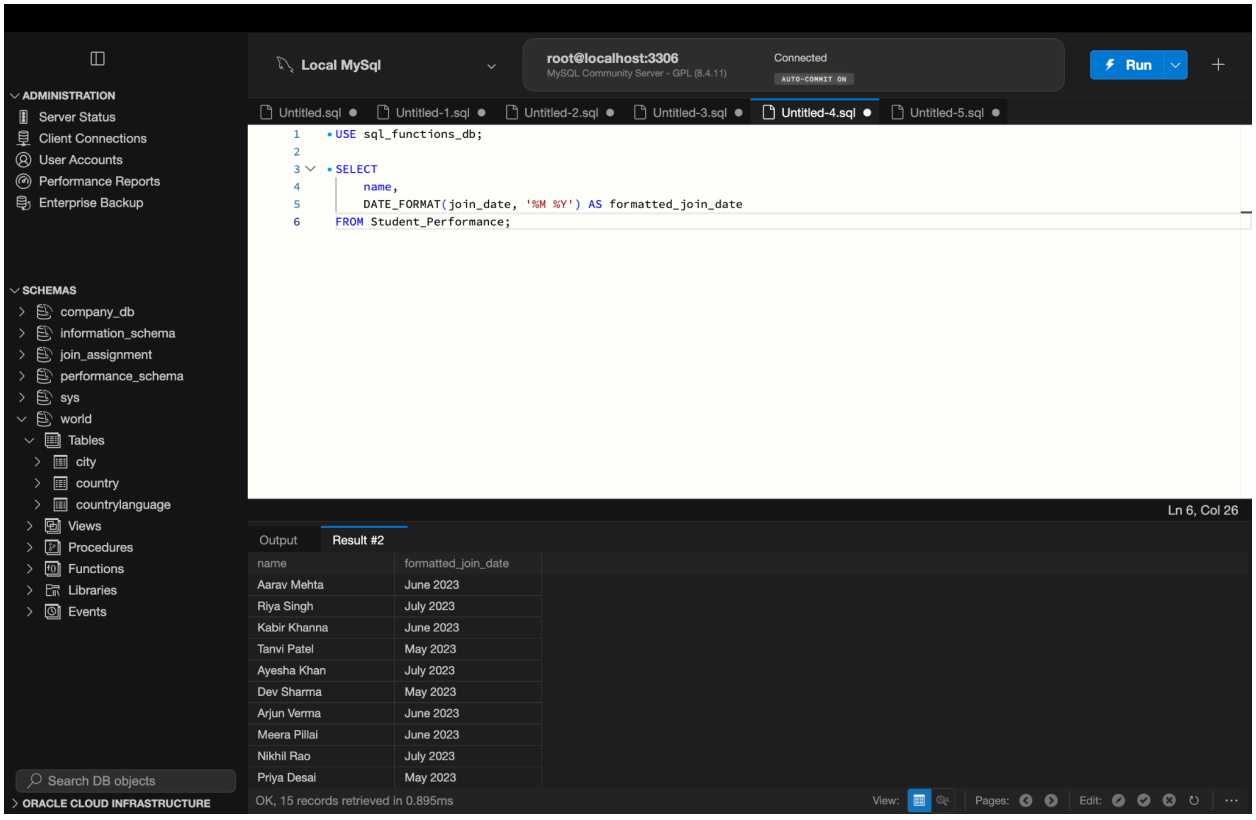

SELECT
    name,
    DATE_FORMAT(join_date, '%M %Y') AS formatted_join_date
FROM Student_Performance;
```

Explanation:

The `DATE_FORMAT()` function formats a date according to a specified pattern. `%M` returns the full month name, and `%Y` returns the four-digit year.

Result: The join dates are displayed in the required Month Year format.

Practical proof :





Question 9: Replace the city ‘Mumbai’ with ‘MUM’ for display purposes.

SQL Query:

USE sql_functions_db;

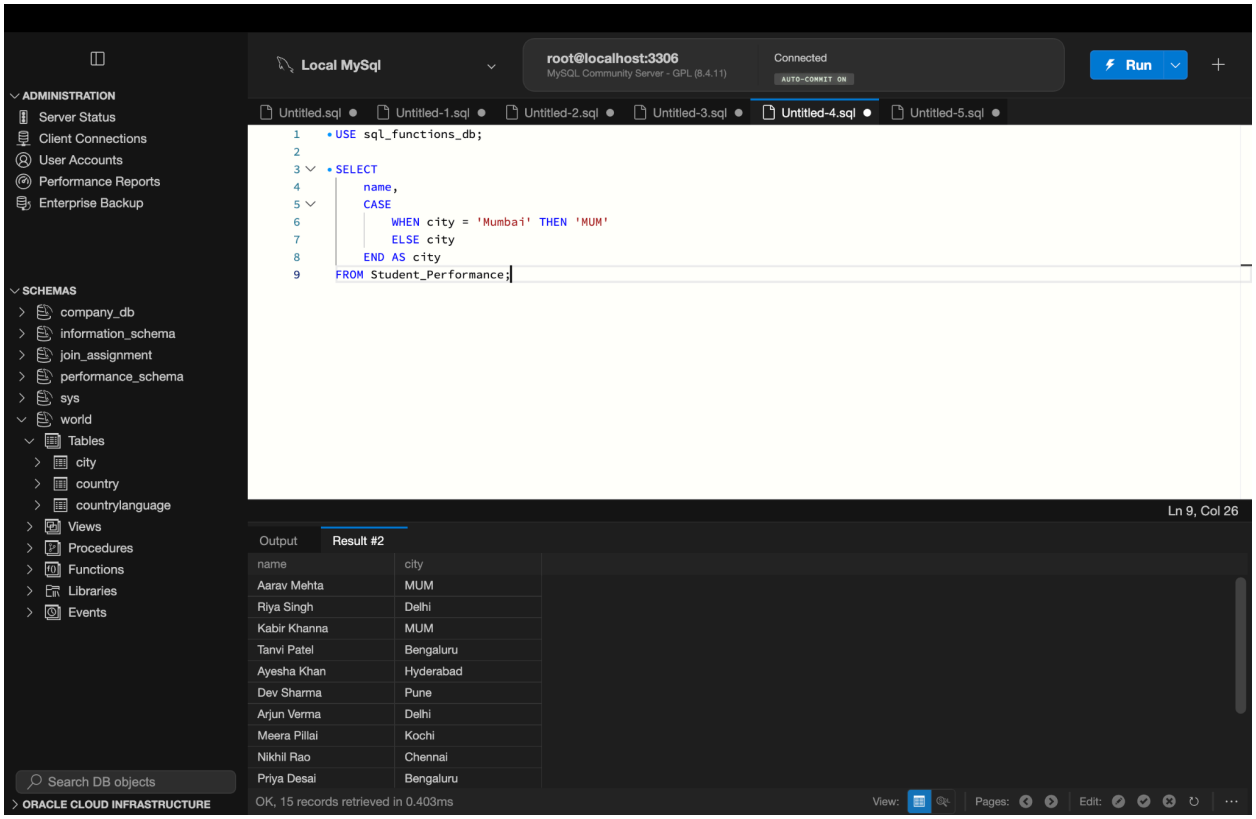
```
SELECT
    name,
    CASE
        WHEN city = 'Mumbai' THEN 'MUM'
        ELSE city
    END AS city
FROM Student_Performance;
```

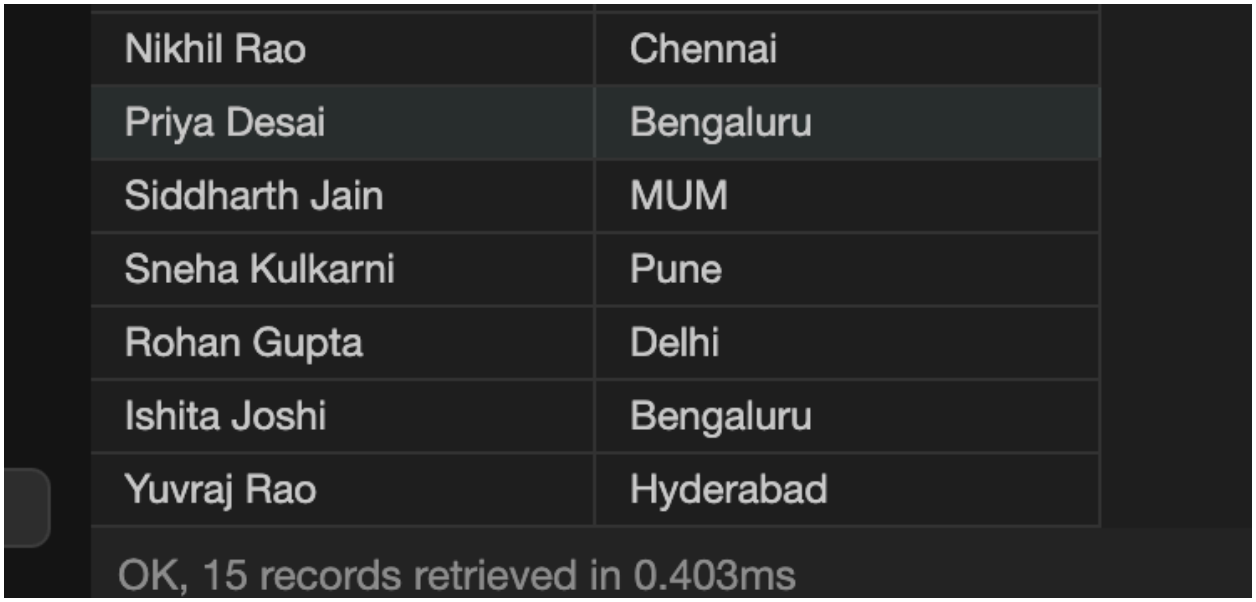
Explanation:

The `CASE` expression checks whether a student's city is Mumbai. If it is, the query displays `MUM`; otherwise, it displays the original city name.

Result: Mumbai is displayed as MUM without modifying the original data in the table.

Practical proof :





Nikhil Rao	Chennai
Priya Desai	Bengaluru
Siddharth Jain	MUM
Sneha Kulkarni	Pune
Rohan Gupta	Delhi
Ishita Joshi	Bengaluru
Yuvraj Rao	Hyderabad

OK, 15 records retrieved in 0.403ms

Question 10: For each course, find the highest score using FIRST_VALUE().

SQL Query:

```
USE sql_functions_db;


SELECT
    name,
    course,
    score,
    FIRST_VALUE(score) OVER (
        PARTITION BY course
        ORDER BY score DESC
    ) AS highest_course_score
FROM Student_Performance
```

ORDER BY course, score DESC;

Explanation:

The `FIRST_VALUE()` function retrieves the first value in each window. `PARTITION BY course` separates students by course, while `ORDER BY score DESC` places the highest score first in each course.

Result: The query displays each student's score alongside the highest score achieved in their course.

Practical proof :

